Expert Systems With Applications 298 (2026) 129851

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

LLM-based exploration and analysis of real-time and historical blockchain data

S. Gebreab [a], A. Musamih [b,*], K. Salah [a], R. Jayaraman [c]

[a] Department of Computer & Information Engineering, Khalifa University, Abu Dhabi, UAE
[b] Department of Management Science & Engineering, Khalifa University, Abu Dhabi, UAE
[c] Department of Industrial Engineering, New Mexico State University, Las Cruces NM MSC 4230, USA

ARTICLE INFO

Keywords:
Blockchain
Blockchain explorers
Large language model (LLM)
Retrieval-augmented generation (RAG)
LLM Agents
Ethereum

ABSTRACT

Blockchain technology has revolutionized digital transactions and decentralized applications through its transparent and immutable ledger system, with platforms like Ethereum processing millions of transactions daily. However, as blockchain networks grow, traditional blockchain explorers show limitations when providing intuitive access to this vast data landscape, particularly when handling complex analytical queries, interpreting transaction patterns, and serving users without technical expertise. In this paper, we address these limitations by proposing an intelligent blockchain explorer that combines a Large Language Model (LLM)-powered agent for real-time blockchain interactions with a schema-aware SQL agent for historical data analysis. For real-time interactions, a dedicated blockchain agent connects to live networks through external APIs and specialized tools to process queries about current transactions and network states. When analyzing historical data patterns, we use an approach in which a Retrieval-Augmented Generation (RAG) system enhances the SQL agent's understanding of the blockchain database schema and structure. This SQL agent subsequently translates natural language queries into SQL commands for efficient data retrieval from our periodically synchronized blockchain database. A query processor, powered by an LLM, intelligently routes user queries between these components based on temporal and contextual requirements, which enables both immediate blockchain state analysis and complex historical data querying. We evaluate our system on diverse blockchain queries, including complex analytical scenarios and multi-step operations. The experimental results demonstrate the effectiveness of our schema-aware SQL agent in accurate query translation and the overall system's capability in handling both real-time and historical blockchain data exploration tasks.

1. Introduction

Blockchain technology has found its application in various industries by offering a decentralized, transparent, and tamper-proof system to record transactions (Monrat et al., 2019; Nakamoto, 2008). Although initially developed to facilitate Bitcoin transactions, blockchain networks like Ethereum have significantly expanded their scope and currently process millions of daily transactions across diverse domains, such as decentralized finance (DeFi), supply chain management, and digital asset ownership. Consequently, the volume of on-chain data has expanded at an exponential rate. This growth in transaction data, combined with the rising analytical demands of various stakeholders, highlights the need for intuitive exploration tools to bridge the gap between raw blockchain data and actionable insights. Despite their utility, traditional blockchain explorers lack intuitive mechanisms for querying complex transaction patterns or deriving meaningful insights from the immense volume of data (Tovanich et al., 2021). Their rigid query interfaces are designed primarily for simple transaction tracking, making it challenging for non-technical users to formulate precise queries, navigate intricate transaction histories, or consolidate information across multiple blocks and accounts. While platforms like Etherscan provide essential functionalities such as basic transaction lookups and network monitoring, they face significant limitations. One major drawback is their inability to process complex, multi-parameter queries. These systems rely on basic filters and search capabilities, which fail to address queries that require contextual understanding or advanced analytical depth. Another issue is the absence of a natural language interface. Users must rely on predefined filters or structured query languages, which are often unintuitive and difficult to use, especially for non-technical individuals.

* Corresponding author.
E-mail addresses: senay.gebreab@ku.ac.ae (S. Gebreab), ahmad.musamih@ku.ac.ae (A. Musamih), khaled.salah@ku.ac.ae (K. Salah), rjayaram@nmsu.edu (R. Jayaraman).

https://doi.org/10.1016/j.eswa.2025.129851
Received 8 April 2025; Received in revised form 12 August 2025; Accepted 21 September 2025
Available online 24 September 2025

This constraint limits accessibility and usability for a broader audience. In addition, traditional blockchain explorers require a thorough understanding of blockchain structures and activities. This reliance on domain-specific expertise creates barriers for both newcomers and experienced analysts attempting to interpret intricate on-chain activities. Lastly, these explorers lack advanced analytical tools. They do not support features such as pattern identification, trend analysis, and predictive insights, which are essential for modern blockchain applications like DeFi.

The rise of Large Language Models (LLMs), including OpenAI's GPT, Google's Gemini, and Meta's LLama, has introduced transformative possibilities for text understanding and generation (Bharathi Mohan et al., 2024; Gallifant et al., 2024; Gemini Team Google, 2024; Touvron et al., 2023). These advanced models, trained on vast datasets, excel in interpreting complex questions, engaging in reasoning, and producing coherent responses (Brown et al., 2020). Despite their capabilities, LLMs face limitations in accessing and processing real-time or domain-specific data. Retrieval-Augmented Generation (RAG) addresses this gap by enabling LLMs to tap into external knowledge bases during query execution (Gao et al., 2024; Lewis et al., 2021). In this approach, the RAG system retrieves relevant information from connected data sources and combines it with the LLM's generative abilities. This integration ensures responses that are not only grounded in factual data but also contextually relevant.

Traditional LLMs excel at processing and generating text but face a significant limitation because they cannot independently interact with external systems or perform actionable tasks. These models cannot execute commands, communicate with APIs, or use external tools, which restricts their utility in scenarios that require direct interaction with real-world systems (Gebreab et al., 2024; Wang et al., 2024). LLM agents resolve this limitation by combining a fine-tuned LLM with a framework that executes programmatic actions. When a user submits a request, the agent analyzes the task, divides it into discrete steps, and produces specific function calls for each action. These calls are executed by the framework, allowing the system to access APIs, retrieve information, or carry out file operations. This architecture converts LLM agents into autonomous systems that handle multi-step tasks while engaging directly with external systems. In addition, the framework enables iterative reasoning through chains of thought and feedback loops. By reviewing the results of its actions, the agent updates its approach and adjusts subsequent steps.

In this paper, we combine these technologies to create an intelligent blockchain exploration system. We use an LLM agent to handle real-time blockchain queries through direct network interactions. This agent can fetch current block data, track live transactions, and monitor network status. For historical analysis, we implement a SQL agent that acts on indexed blockchain data, which is periodically synchronized with the network. The SQL agent is enhanced by a RAG system that gives it access to comprehensive knowledge about the database, including its schema, table relationships, common query patterns, and example SQL commands. This knowledge enables the agent to accurately translate natural language questions into SQL queries tailored to our database structure. A query processor determines whether to route requests to the real-time agent or the historical analysis component based on the temporal nature of the query. This dual approach lets users explore both current and historical blockchain data through simple natural language queries, without needing technical expertise in blockchain or SQL. The main contributions of this paper can be summarized as follows.

- We develop an intelligent LLM-powered blockchain explorer that combines real-time blockchain interaction with sophisticated historical data analysis through a natural language interface.
- We implement schema-aware SQL agent architecture that leverages RAG capabilities to understand database structure and translate natural language queries into precise SQL commands for blockchain data retrieval.
- We introduce an LLM-powered blockchain agent equipped with blockchain-specific tools and APIs for real-time blockchain network interaction and state analysis.
- We implement an ETL pipeline and indexing system that periodically synchronizes and optimizes blockchain data for efficient querying.
- We add a dynamic generative UI that not only translates complex blockchain data and query results into natural language responses, but also generates UI elements, including table and charts, to present relevant data.

The remainder of this paper is organized as follows. Section 2 introduces the foundational concepts of blockchain technology, LLMs, and existing blockchain exploration tools. Section 3 presents the related work. Section 4 details the proposed system, including its architecture and core components. Section 5 describes the implementation details of the solution, including the tools and technologies used. Section 6 presents our experimental setup and evaluation results, followed by a discussion of the findings. Finally, Section 7 concludes the paper and outlines potential directions for future work.

2. Preliminaries

This section lays the foundation for this paper by describing key concepts and technologies, including blockchain technology, LLMs, RAG, and blockchain exploration tools.

2.1. Blockchain technology

Blockchain technology is a decentralized, distributed ledger system that records transactions across a network of computers (Nakamoto, 2008). It is characterized by its immutability, transparency, and security, which is achieved through cryptographic hashing and consensus mechanisms (Monrat et al., 2019; Xu et al., 2019). Each block in the chain contains a set of transactions, a timestamp, and a reference to the previous block, forming a chronological and tamper-resistant record of all network activities.

Ethereum, introduced in 2015 (Buterin, 2014; Wood, 2014), extended the concept of blockchain beyond simple financial transactions by incorporating smart contracts, which are self-executing programs that run on the blockchain. This innovation has led to a wide range of decentralized applications (dApps) and spawned the entire DeFi ecosystem (Meyer et al., 2022; Werner et al., 2023). As a result, the Ethereum blockchain contains not just transaction data, but also complex contract interactions and state changes, making data exploration and analysis both more crucial and more challenging.

2.2. Large language models (LLMs)

LLMs are advanced artificial intelligence (AI) systems trained on vast amounts of text data (Brown et al., 2020; Chowdhery et al., 2023). These models, such as GPT-4 and Claude 3.5, have demonstrated remarkable capabilities in natural language understanding, generation, and task completion. LLMs can process and generate human-like text, understand context, and perform a wide range of language-related tasks, including question-answering, summarization, and code generation.

The application of LLMs in data exploration and analysis has gained significant traction due to their ability to handle complex queries, interpret context, and generate informative responses (Zhao et al., 2025). Within the blockchain domain, LLMs have the potential to bridge the gap between human-language queries and the technical complexities of blockchain data structures. This capability enables users, regardless of their technical background, to interact with and extract insights from blockchain data in a more intuitive and accessible manner. However, LLMs face limitations in accessing real-time or domain-specific data, which can hinder their ability to provide accurate and contextually relevant insights. Addressing these challenges is crucial to fully unlocking their potential for blockchain applications.

Table 1
Comparative overview of LLM-blockchain integration approaches.

Approach	Focus∗	Main Contribution	Data Freshness	Interface
Bouchiha et al. (2024)	Blockchain → LLM (trust layer)	Immutable reputation ledger that logs prompts, answers, and evaluation scores on-chain	Not applicable	On-chain smart-contract calls
Luo et al. (2023)	Blockchain → LLM (audit trail)	Conceptual framework to store every stage of an LLM's lifecycle (training data, parameters, inference I/O) on-chain for provenance	Not applicable	Not implemented
Gai et al. (2023)	LLM → Blockchain (analytics)	Domain-specific transformer that flags anomalous Ethereum transactions in real time	Live (RPC stream)	API
Xian et al. (2024)	LLM → Blockchain (oracle)	Oracle framework that fuses multiple LLM answers using semantic consensus to feed smart contracts	Near-real-time (oracle feeds)	JSON oracle API
ChainGPT (2025)	LLM → Blockchain (developer tools)	Chatbot that can pull live on-chain facts, audit Solidity, and draft transactions	Live (multi-chain RPC)	Web chat & SDK
Our Work	LLM → Blockchain (unified explorer)	Intelligent blockchain explorer combining live tool-calling agent and RAG-augmented natural language to SQL pipeline for historical data analysis	Live (RPC + indexed historical DB)	Chat interface with generative analytics UI

∗. *Blockchain → LLM* indicates blockchain is used for LLM transparency/provenance; *LLM → Blockchain* indicates LLM is used for blockchain analytics or tooling.

2.3. Retrieval-augmented generation (RAG)

RAG is a hybrid approach that combines the strengths of retrieval-based systems with generative AI models (Guu et al., 2020; Izacard & Grave, 2021; Lewis et al., 2021). In a RAG system, a retrieval component first retrieves relevant information from a knowledge base, which is then used to augment the input to a generative model. This integration enables the system to access extensive external knowledge, while allowing the generative model to produce coherent and detailed outputs.

In the context of blockchain exploration, RAG systems offer the potential to efficiently retrieve historical transaction data and blockchain states, which can then be used to inform and enhance LLM-generated responses to user queries. This combination can lead to more accurate, contextually relevant, and up-to-date information provision, particularly for queries that require access to specific historical data points or patterns.

2.4. Blockchain exploration tools

Traditional blockchain explorers, such as Etherscan for Ethereum, provide web-based interfaces for users to view transaction histories, account balances, and smart contract interactions. These tools typically offer search functionality based on transaction hashes, account addresses, or block numbers. Although they provide essential information, they often require users to have technical knowledge of blockchain structures and manually navigate through complex transaction trails. Current blockchain exploration tools face several limitations:

- Limited support for natural language queries, which forces users to create technically precise search parameters.
- Difficulty in extracting high-level insights or patterns from large volumes of transaction data.
- Lack of adaptive query processing mechanisms to handle diverse query types efficiently.
- Challenges in synthesizing information across multiple transactions or accounts for complex analyses.

These limitations highlight the need for more advanced, user-friendly, and intelligent blockchain exploration systems.

3. Literature review

This section reviews key studies at the intersection of blockchain technology and LLMs, which focus on their potential to address limitations in traditional blockchain explorers. A summary of these studies is presented in Table 1.

The integration of LLMs into blockchain technology has gained traction as researchers explore avenues to enhance trust, transparency, and interpretability in LLM-driven systems. Bouchiha et al. (2024) introduce LLMChain, a blockchain-based framework designed to establish a reputation system for LLMs, where blockchain's decentralized nature is used to assess and share models' reputations. The authors argue that blockchain's transparent and immutable characteristics could address some trustworthiness issues related to LLMs, particularly in critical decision-making scenarios where model credibility is paramount. However, the framework primarily focuses on model assessment and reputation within the ecosystem and lacks empirical results on the framework's scalability and effectiveness in real-world applications. Additionally, LLMChain does not directly address the complexities involved in natural language querying for blockchain data, leaving a gap in its application for user-facing blockchain exploration.

Similarly, Luo et al. (2023) present BC4LLM, a framework focused on enhancing AI trustworthiness by merging LLMs and blockchain. This integration seeks to mitigate security and reliability concerns through blockchain's verifiable audit trails. BC4LLM addresses the challenges of

data tampering and unauthorized access within LLM applications, underscoring the potential of blockchain to enforce accountability in the storage, usage, and interpretation of language model outputs. However, the framework does not provide a detailed implementation strategy for integrating this trusted AI model into blockchain explorers. Furthermore, it does not address how BC4LLM could be extended to facilitate user-centered natural language interactions, which are essential for an intelligent blockchain explorer.

Xian et al. (2024) propose a framework called C-LLM for integrating LLMs with blockchain data by using semantic relatedness and truth discovery techniques to aggregate LLM-generated data from multiple oracle nodes. This approach is presented to address the interoperability challenge between LLMs and blockchain, as well as data inconsistency during information retrieval. However, the focus of the paper is on data reliability and does not extend to on time-bound blockchain data analysis nor user accessibility.

There have been several studies that propose the use of blockchain to address some of the security and safety challenges associated with LLMs. In their survey, Geren et al. (2024) evaluate blockchain's potential to mitigate vulnerabilities in LLMs, including issues of model spoofing, injection attacks, and hallucinations in response generation. The study highlights blockchain's capability to enhance LLM security by providing immutable storage and verifiable access logs, which could play an essential role in protecting LLMs used in blockchain explorers from adversarial manipulation. However, while Geren et al. provide valuable insights into security mechanisms, they do not explore practical, user-facing implementations of LLMs in blockchain systems. Similarly, He et al. (2025) offer a comprehensive review of LLM-driven applications in blockchain security. Their work explores LLM-driven applications such as smart contract auditing and anomaly detection. While this review provides a solid theoretical background for security applications, it does not address user-centric data exploration. Toyoda et al. (2024) present a broad survey of LLM-integrated blockchain data analysis providing a comprehensive look at the challenges and opportunities for LLMs in blockchain analysis. However, the survey lacks details on how to implement an LLM-based blockchain explorer that combines real-time and historical data analysis.

The application of LLMs for intelligent blockchain explorers is further expanded by Gai et al. (2023) with their presentation of BlockGPT, an LLM-based tool designed to perform real-time anomaly detection within blockchain transactions. BlockGPT demonstrates that LLMs are capable of sophisticated data interpretation in real-time, facilitating anomaly detection, transaction analysis, and user-friendly exploration. While BlockGPT exemplifies a new class of blockchain tools where LLMs' natural language understanding abilities can bridge the gap between complex blockchain data and non-expert users, the tool remains limited in its ability to perform diverse types of queries beyond anomaly detection.

4. Proposed system

The proposed LLM-powered blockchain explorer is designed to simplify the way users interact with and extract information from blockchain networks. In traditional blockchain explorers users need to be familiarized with the platform and often are required to perform multiple steps to retrieve information. Moreover, these platforms suffer from limited contextual understanding, and cannot handle natural language queries. Our proposed system addresses these limitations by integrating LLM-based agents, and a RAG framework. By combining these technologies, we allow the blockchain explorer to provide a more intuitive, efficient, and context-aware experience.

The system is designed to manage both real-time and historical blockchain data interactions using a dual-mode architecture. The system's processing capabilities are divided into two primary pathways: Online Blockchain Processing for real-time interactions and an Offline Processor for historical data analysis. The Offline Processor uses a schema-aware LLM-powered SQL agent, enhanced by a RAG framework, which operates on a periodically synchronized relational database of indexed blockchain data. The Online Blockchain Processor uses an LLM-based blockchain agent for real-time interaction with the blockchain network through APIs and other tools. This dual-mode approach is important because relying only on one method would create limitations in the system. A purely online approach would introduce performance and scalability challenges. Some complex queries would need to retrieve and process large amounts of historical blockchain data through real-time calls, which would lead to high delays and possible rate limits on the API if blockchain node providers' services are being used. The blockchain's design as a transactional ledger rather than an analytical database means that complex queries often require scanning multiple blocks and reconstructing historical states, which are impractical to do in real-time. In contrast, using only the offline-mode approach takes away from the system the flexibility for queries and tasks that need access to the current state of smart contracts, pending transactions, and real-time network data. Keeping a real-time index of all blockchain data to solve this issue has its own complexities and would lead to synchronization delays and potential inconsistencies.

At the core of the online blockchain processing mode, the system uses an LLM-powered agent that can understand and answer natural language questions in real-time. This agent has access to tools and APIs that enable it to interact with the blockchain network through a gateway and perform complex operations such as transaction tracing, smart contract analysis, and retrieving real-time data. At the same time, the Offline Mode combines a SQL agent enhanced by a RAG system and an Extract, Transform, Load (ETL) pipeline for indexing blockchain data. The SQL agent's primary role is to translate natural language queries into precise SQL commands for data retrieval (Pourreza et al., 2024; Shi et al., 2024). The RAG system plays a role in this process by providing the SQL agent with an understanding of the database schema and structure. The ETL pipeline takes raw blockchain data and turns it into a structured format that is easy to filter or analyze for historical trends and patterns. This dual-mode architecture ensures that the system can handle both real-time questions and detailed historical data analysis efficiently and accurately. Mathematically, the system can be represented as a function S that maps user queries Q to responses R, where $S(Q) = R$. This function is defined as shown in Eq. (1):

$$S(Q) = \begin{cases} f_{\text{online}}(Q) & \text{if } Q \in \text{Real-time queries} \\ f_{\text{offline}}(Q) & \text{if } Q \in \text{Historical data queries} \end{cases} \tag{1}$$

Here, f_{online} and f_{offline} denote the processing functions of the Online and Offline Modes, respectively. The system's modular architecture allows it to adapt as user queries become more complex.

Fig. 1 illustrates the high-level architecture of the LLM-powered blockchain explorer, which consists of five main components: the Dynamic User Interface, Query Processor, Online Modes (OM) Processor, Offline Mode (OFM) Processor, and an LLM.

- **Dynamic User Interface**: Provides a natural language input interface for users to submit their queries about blockchain data. It also adaptively renders appropriate UI elements to display response to user queries, including tables and charts.
- **OM Processor**: Handles real-time interactions with the blockchain network for up-to-date information.
- **OFM Processor**: Utilizes a RAG system to efficiently retrieve and process historical blockchain data.
- **Query Processor**: Classifies incoming queries and determines the most appropriate processing mode.
- **LLM**: Acts as the brain for the agents involved in the online and offline data processing.

4.1. Online mode (OM) processor

The OM Processor is designed to handle queries that require real-time interaction with the blockchain. This mode is useful for getting the

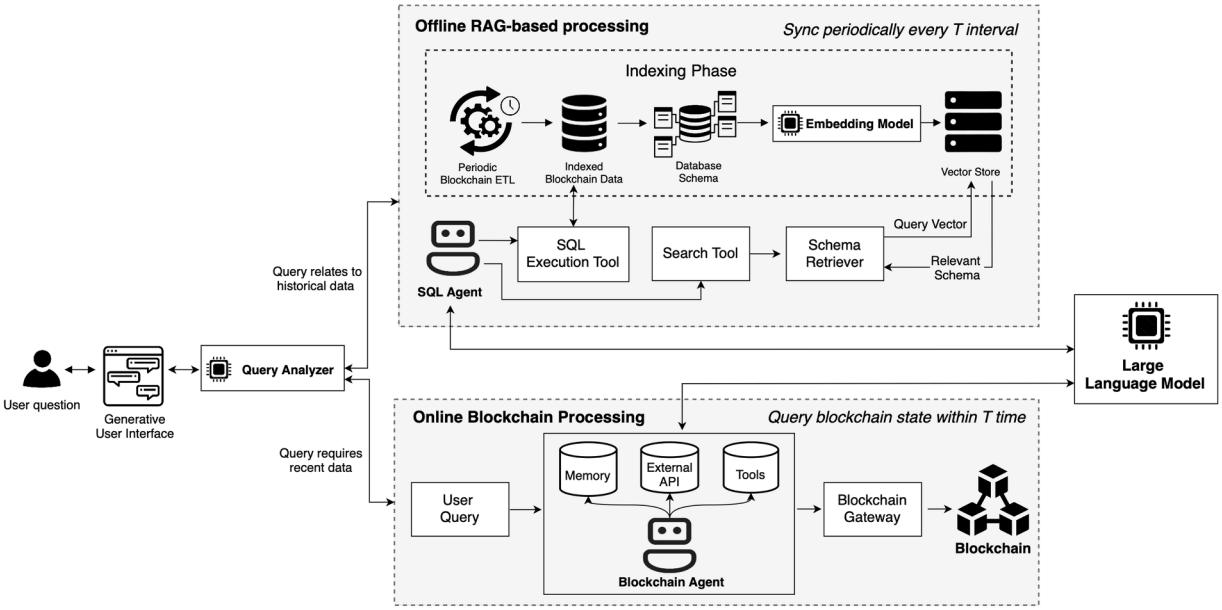


Fig. 1. System architecture of the LLM-powered blockchain explorer showing the dual-mode processing system: (1) Online mode featuring a blockchain agent with real-time API access for current state queries, and (2) Offline Mode utilizing a RAG-enhanced SQL agent for historical analysis.

latest data about recent transactions, current account balances, or live network statistics. The core of the OM is an agent powered by an LLM and equipped with short-term memory for maintaining query context, integration with external APIs and tools for blockchain network access, transaction analysis and state queries. It acts as an intermediary between the user and the blockchain, capable of understanding complex queries, determining appropriate actions, and interpreting blockchain data in context. It can perform multi-step tasks such as:

- **Transaction Tracing**: Following the flow of transactions across multiple blocks and addresses.
- **Smart Contract Analysis**: Evaluating the functionality and behavior of deployed smart contracts.
- **Real-Time Data Retrieval**: Fetching the latest blockchain data, including new blocks, transactions, and network statistics.

To power its reasoning and tool use capabilities, the agent is connected to an LLM, such as GPT-4o or Claude sonnet 3.5, that acts as the main brain. In addition, this agent is provided access to a set of tools, including JSON-RPC API functions, that allow it to directly interact with the blockchain network. When a query is classified for OM processing, the OM agent first analyzes the user's query to understand the intent and identify the required blockchain interactions. Then, based on the query analysis, the agent formulates a plan of action, chooses the right tools and APIs, and then executes the actions, which might involve multiple API calls. As the agent receives data from its tools and API calls, it interprets this information in the context of the original query. Finally, the agent takes the interpreted data and gives out a response that directly addresses the user's query. The agent, therefore, generates a response conditioned on both the query Q and the current state of the blockchain B_t. The agent's ability to perform complex operations can be formalized as Eq. (2), where R_t represents the real-time response at time t. This process allows the OM to handle a wide range of complex queries, from simple balance checks to intricate multi-step operations. However, while the LLM-powered agent offers flexibility, it may be relatively slow for queries that require extensive filtering or searching of transactions, as blockchain data accessed through gateways is not inherently searchable.

$$R_t = \text{Agent}_{\text{LLM}}(Q, B_t) \tag{2}$$

4.2. Offline mode (OFM) processor

The OFM Processor is responsible for handling queries related to historical blockchain data and complex analytical tasks. By utilizing a RAG system, OFM allows the retrieving and processing of large volumes of pre-indexed blockchain data with speed and accuracy. This mode is ideal for answering queries that require aggregated statistics, historical trend analysis, or detailed exploration of past blockchain activity.

At the heart of the Offline Mode is a RAG system that combines the strengths of information retrieval and language generation. The RAG architecture consists of three main components:

- **Data Indexing and Storage**: Historical blockchain data is periodically extracted, processed, and stored in a PostgreSQL database. PostgreSQL is an open-source relational database management system that emphasizes extensibility and SQL compliance (Juba et al., 2015). The stored data blockchain data is then indexed using advanced embedding techniques to enable fast and efficient retrieval.
- **Retrieval Module**: When a query is processed in OFM, the retrieval module uses a dense vector similarity search to identify the most relevant pieces of information from the indexed database.
- **Generation Module**: The retrieved information is then passed to an LLM, which generates a comprehensive and contextually relevant response based on the original query and the retrieved data.

In OFM, the methodology employs a comprehensive ETL pipeline to preprocess and index historical blockchain data. The extraction phase involves collecting raw transaction data $T = \{t_1, t_2, \ldots, t_n\}$ from the blockchain network. The transformation phase processes this data into a structured format T', applying normalization and enrichment techniques to enhance data quality. The loading phase then indexes T' into a searchable database D, facilitating efficient retrieval operations. The RAG system integrates this indexed dataset with the LLM to perform deep historical analysis. The retrieval component R selects relevant documents D_r from D based on the query Q, and the generation component G, which is an LLM model, synthesizes the final response R as:

$$R = G_{\text{LLM}}(Q, R(Q, D)) \tag{3}$$

4.3. Query processor

The Query Processor is the component that is responsible for routing queries to the appropriate processing mode. It makes use of the reasoning capabilities of an LLM to analyze incoming user queries and determine the most appropriate processing mode or combination of modes to efficiently and accurately respond to the user's request. Its decision-making process is primarily driven by the temporal analysis of query requirements in relation to the last synchronization time of the indexed blockchain data and the processing time window of the OM. The LLM model used for the Query Analayzer is prompted to consider several factors to determine the optimal processing approach. For a given query, it analyzes several aspects, including:

- **Temporal relevance**: Determining if the query relates to recent or historical blockchain data.
- **Complexity**: Assessing whether the query requires simple data retrieval or complex analysis.
- **Data scope**: Identifying if the query involves a specific transaction, address, or broader blockchain statistics.
- **Contextual requirements**: Evaluating the need for additional context or background information to provide a comprehensive response.

A Hybrid Mode is particularly crucial for complex analytical queries that require data correlation across different time periods. For example, a query like "Compare the average gas prices from last year to the current rates" would necessitate both historical data from the indexed database and real-time data from the blockchain. In such cases, the Query Processor orchestrates parallel processing across both modes and coordinates the aggregation of results. The query processor is also responsible for formatting and presenting the final output to the user. It takes the raw results from the OM and/or OFM processors and uses an LLM to generate a coherent, natural language response that addresses the user's query.

5. Implementation details

In this section, we outline the implementation details of the ETL pipeline, the RAG system, and the LLM agent component.

5.1. Experimental setup

The experimental evaluation was conducted on the Ethereum mainnet. We chose Ethereum because it is a leading smart contract platform with extensive DeFi activity and rich transaction data. We used Infura's API endpoints for blockchain interaction. The software stack included PostgreSQL 13.4 for data storage, with LLM integrations comprising GPT-4o, Claude-3.5-Sonnet, and Gemini-2.0-Flash accessed through their respective APIs. FAISS handled embedding storage and similarity search, while the ETL pipeline was implemented using Python's asyncio framework. OpenAI's text-embedding-3-small model handled metadata chunking for the RAG system.

5.2. Online mode implementation

The OM component allows real-time access to the latest state of the blockchain network. It includes a blockchain interface layer that manages connection to the Ethereum network. To implement this interface, we use Web3.py library, which comes fully supporting Ethereum's JSON-RPC API. The interface layer uses multiple node providers where Infura serves as the primary endpoint, while additional backup providers such as Alchemy and QuickNode can be optionally configured for failover scenarios, improving uptime. The system maintains a live chain time window (T) that defines the recent blockchain state accessible through the Online Mode. This window represents the most recent blocks that can be queried directly from the blockchain network in real-time that are within the time-frame T. This window T is a configurable parameter that can be set based on application requirements, network resources. Setting T to 1.5 days for Ethereum spans approximately the last 10,000 blocks. Any queries requesting data within time window T are handled by the Online Mode, allowing direct interaction with the blockchain network for current information.

The blockchain agent is designed for handling the blockchain interaction and analysis (Algorithm 1). The agent uses a combination of LLM capabilities, blockchain-specific tools, and decision-making methods to execute complex blockchain operations and analyses (Wei et al., 2022; Zhu et al., 2023). It has access to a set of tools that implement blockchain operations through modular function calling, defined using JSON-schema specifications. Each tool handles specific blockchain functions, such as checking account balances or querying smart contract data, and includes error handling and response validation to ensure that the process runs smoothly.

Algorithm 1: LLM-powered agent for real-time blockchain interactions.

Input: Natural language query Q, Current blockchain state B_t, LLM model, External APIs
Output: Real-time response R_t
begin
  Step 1: Initial Reasoning (Chain-of-Thought)
    $thought \leftarrow$ LLM generates initial reasoning based on (Q, B_t);
  Step 2: Action Planning
    $actions \leftarrow$ LLM identifies blockchain interactions from $thought$;
  Step 2: Tool Invocation
    foreach *action in* $actions$ **do**
      if *external information required* **then**
        $tool\_output \leftarrow$ `CallTool` ($ExeternalAPIs$);
        $thought \leftarrow$ integrate $tool\_output$ into $thought$;
      else
        continue without external calls;
      end
  end
  Step 3: Response Generation
    $R_t \leftarrow$ LLM synthesizes final response from updated $thought$ and context B_t;
  Step 4: Memory Update
    `UpdateMemory` (Q, R_t);
  Step 5: Format and Return Response
    Format R_t for presentation to user;
    return R_t;
end
Function `UpdateMemory` *(*Q, R_t*)*:
  Store query-response pair (Q, R_t) for future context retrieval;

5.3. Ethereum ETL and indexing

The Ethereum ETL process forms a critical component of our LLM-powered blockchain explorer. This subsection details the architecture and implementation of our ETL pipeline, which is designed to efficiently extract and index Ethereum blockchain data for subsequent use in our RAG system. The ETL process operates on a regular sync period (T), which determines how frequently the system updates its indexed database with new blockchain data.

Our ETL pipeline is built on a robust, asynchronous architecture to handle the high volume of blockchain data efficiently. The system comprises several key components:

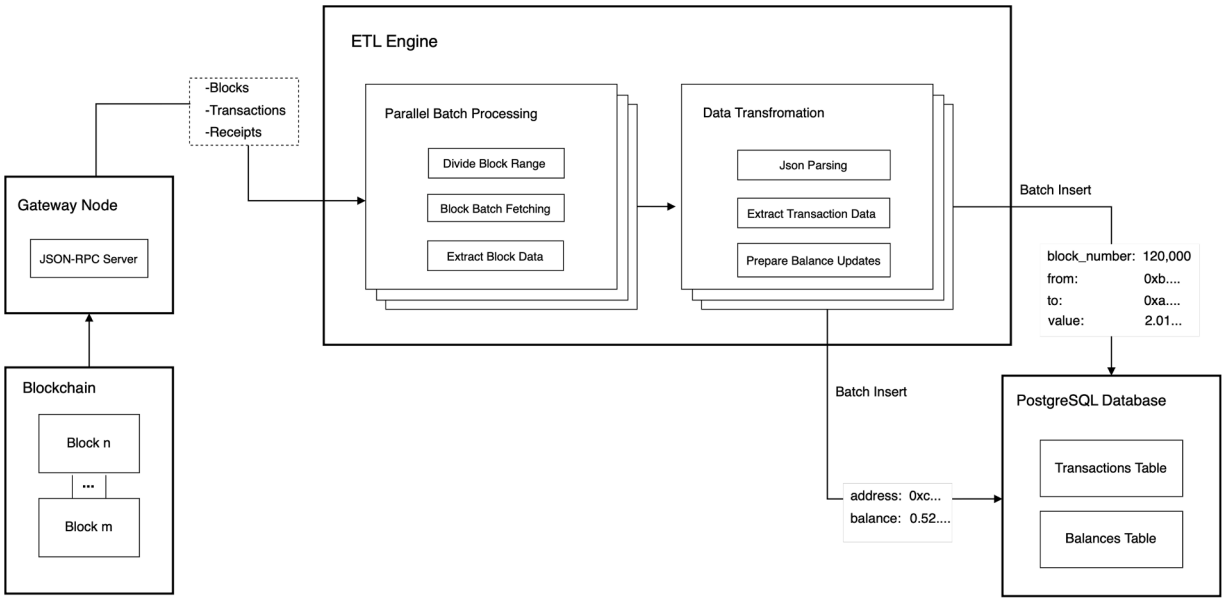


Fig. 2. Overview of the Ethereum ETL and indexing pipeline.

1. **Data Source**: We primarily utilize the Ethereum Mainnet, accessed through JSON-RPC API providers such as Infura [1] and PublicNode [2] or through a local GETH node.
2. **ETL and Indexing Engine**: The core of our system is implemented in Python, centered around the *EthereumIndexer* class. This component leverages key libraries including:
   - `asyncio` for asynchronous programming
   - `web3.py` for Ethereum blockchain interactions
   - `psycopg2` for PostgreSQL database operations
3. **Data Storage**: We use PostgreSQL as our primary data store, with two main tables:
   - `transactions`: Stores detailed transaction information
   - `balances`: Maintains up-to-date account balances

Our Ethereum ETL process is designed as an end-to-end pipeline that efficiently extracts, transforms, and loads blockchain data. As shown in Fig. 2, the process begins with the selection of a block range. Once the block range is determined, our system initiates the data extraction phase. Leveraging the asynchronous capabilities of Python's asyncio library, we establish concurrent connections to the Ethereum network via JSON-RPC API providers. This parallelized approach allows us to fetch multiple blocks simultaneously, significantly reducing the overall time required for data retrieval. As each block is fetched, we immediately begin extracting pertinent transaction information, including transaction hashes, sender and recipient addresses, transaction values, and associated block numbers. This real-time extraction minimizes memory overhead and allows for efficient processing of large datasets.

The transformation phase of our ETL pipeline is where raw blockchain data is refined into a structured format suitable for database storage and subsequent analysis. A key aspect of this transformation is the conversion of transaction values and account balances from Wei (the smallest unit of Ether) to Ether. This conversion helps in improving the readability and usability of the data for both human analysts and AI models. Additionally, our transformation process includes data normalization steps, such as standardizing address formats and handling potential inconsistencies in the raw data. We also enrich the data by deriving additional metrics and attributes that may be valuable for analysis, such as calculating the gas used for each transaction or flagging specific types of transactions (e.g., contract creations or token transfers). The final stage of our ETL process is data loading, where the transformed data is efficiently inserted into our PostgreSQL database. We employ a bulk insertion strategy to minimize database round-trips and optimize write performance. For transaction data, we utilize a carefully crafted upsert operation that allows us to handle potential reorgs in the Ethereum blockchain gracefully. This approach ensures that our database always reflects the most up-to-date state of the blockchain, even in the face of network disruptions or temporary forks. Concurrently with transaction insertion, we update the balances table to maintain an accurate snapshot of account states. This balance tracking is crucial for enabling fast and accurate responses to balance-related queries without the need for resource-intensive aggregations at query time. The ETL process is performed periodically and syncs updates incrementally, with only newly added blocks being indexed.

At the heart of the ETL pipeline lies the `index_block_range` method of the `EthereumIndexer` class. Rather than ingesting an entire block interval at once, this method slices the range into configurable batches. In our experiments, we set the batch size to 100 blocks. On a MacBook Pro with an M1 chip (10-core CPU, 16 GB RAM), using PublicNodes's RPC endpoint, a 1000-block range processed in batches of 100 completes the full ETL loop in an average of 298.39 s. The job is triggered every 60 min to synchronize the Ethereum blockchain data with the indexed database. Each batch is then handed off to the `process_block_batch` method, the pipeline's workhorse. This routine spawns asynchronous tasks, one per block, to overlap I/O-bound JSON-RPC calls with CPU-bound transformations. As blocks arrive, the method extracts transaction records, accumulates them in memory, and collects the unique addresses needed for downstream balance updates. Queries that reference blocks newer than the most recent hourly snapshot are automatically routed to the online tool-calling agent, preserving sub-second freshness for real-time lookups without incurring continuous indexing overhead.

The database operations are optimized as well, to keep throughput high and maintain state consistency. We use the *execute_batch* method provided by psycopg2 library, which allows us to send multiple SQL commands to the database server in a single request. This batching significantly reduces network overhead and improves overall insertion speed. For both transaction and balance data, we employ sophisticated upsert logic using PostgreSQL's *"INSERT …ON CONFLICT …DO UPDATE"* syntax. This ensures data consistency even when reprocessing blocks or handling chain reorganizations. In the case of transactions,

[1] https://www.infura.io/
[2] https://ethereum.publicnode.com

we update all fields if a duplicate hash is encountered, reflecting the latest state of the transaction. For balances, we simply update the balance value to ensure that our database always reflects the most recent account state.

For performance optimization we keep the ETL runtime fully asynchronous. By using `asyncio` we maintain dozens of concurrent JSON-RPC sessions, hiding network latency and keeping all cores busy. Batch size remains a adjustable. 100 – 500 blocks gave the best trade-off between memory footprint, write frequency, and overall throughput on our test hardware, but the value can be dialled up or down to match other environments. Finally, targeted B-tree indexes on `from_address` and `to_address` accelerate the address-centric look-ups that dominate our explorer's workload, providing substantial query-time gains at a modest upkeep cost during ingestion.

Algorithm 2: RAG system for historical data retrieval.

Input: Historical query Q, Indexed vector dataset D, Embedding model, LLM model
Output: Generated response R

begin
  1. Data Ingestion Phase
    Knowledge Source Location and Preprocessing;
      $RawData \leftarrow$ `CollectData` $(DataSources)$
      $Chunks \leftarrow$ `PreprocessData`$(RawData)$
    Embedding and Storage;
      $Vectors \leftarrow$ `EmbedChunks`$(Chunks, EmbeddingModel)$
      `StoreVectors`$(Vectors, Chunks, VectorDatabase)$
  2. User question Handling Phase;
    Question Parsing and Embedding;
      $parsedQuestion \leftarrow$ `ParseQuestion`(Q);
      $QuestionEmbedding \leftarrow$ `EmbedQuestion`$(parsedQuestion, EmbeddingModel)$;
    Semantic Search;
      $RelevantChunks \leftarrow$ `SemanticSearch`$(QuestionEmbedding, VectorDatabase)$;
    Response Generation;
      $augmentedPrompt \leftarrow$ Concatenate$(parsed_Q, RelevantChunks)$;
      $R \leftarrow$ `GenerateResponse`$(augmentedPrompt, LLM)$;
  return R;
end

5.4. Offline mode with RAG implementation

The OFM Processor implements a RAG system that combines hierarchical document chunking, semantic search, and context-aware prompt engineering. The implementation follows the process described in Algorithm 2, which details the data ingestion phase and query handling phase for historical data retrieval. For indexing, the database metadata document, which includes schema definitions, query patterns, and optimization guidelines, is structured using hierarchical markers, special characters, for efficient segmentation and retrieval. Fig. 3 illustrates how the document is processed for RAG. For each incoming user query q_{user}, the system first parses and reformulates it into one or more targeted sub-questions designed specifically to retrieve relevant information from the metadata. This step is necessary because the answer to the user query itself does not directly appear in the metadata. The retrieved content only provides schema descriptions, usage patterns, and operational hints that enable the downstream SQL agent to construct a precise SQL query. That SQL query is then executed against the blockchain database to obtain the final answer for the user. To perform the parsing and transformation of the user query, we use an LLM, GPT-4o.

All the steps involved in the RAG process are as follows:

1. **Metadata Chunking**: The system segments the documentation using a hierarchical approach.

   Let D be the document. Applying split(D) produces a set of chunks:

$$C = \{c_1, c_2, \ldots, c_n\} \tag{4}$$

   where each chunk c_i corresponds to a distinct section of the original document (e.g., schema, patterns, hints).
2. **Chunk Processing**: Each chunk is structured into a `MetadataChunk` object:

$$M_i = \{\text{content}, \text{category}, \text{subcategory}, \text{ref\_object}\} \tag{5}$$

3. **Vector Embedding**: Each chunk is embedded using OpenAI's `text-embedding-3-small` model:

$$E_i = f_{\text{embed}}(M_i.\text{content}) \tag{6}$$

4. **Context Retrieval**: For a given query q, the most relevant chunks are retrieved using FAISS:

$$R_k = \arg\max_{c_i \in C} \left(\frac{f_{\text{embed}}(q) \cdot E_i}{\|f_{\text{embed}}(q)\| \, \|E_i\|} \right) \tag{7}$$

   where R_k denotes the top-k most relevant chunks.
5. **Reranking**: The initial retrieval results are refined using a reranking model:

$$R_k = \text{rerank}(R_{\text{initial}}, q, \text{top\_n} = 10) \tag{8}$$

   where `rerank` is Cohere's reranking function (using the `rerank-english-v3.0` model), which reorders chunks based on their relevance to the query.
6. **Context Organization**: Retrieved chunks are grouped by category:

$$O = \{\text{schema} : R_s, \text{patterns} : R_p, \text{hints} : R_h, \ldots\} \tag{9}$$

7. **Prompt Construction**: The final prompt is constructed by combining the query and the organized context:

$$P = \text{concat}(\text{prefix}, O, q, \text{instructions}) \tag{10}$$

The hierarchical chunking strategy proved particularly effective in our implementation. When working with blockchain queries, we found that related concepts often needed to be retrieved together. For instance, a query about transaction values would require not just schema information about the value column, but also context about Wei-to-ETH conversion patterns and numerical precision handling. For embedding model, text-embedding-3-small, which we accessed through OpenAI's API, was the best balance of accuracy and performance for our use case (Kamalloo et al., 2023). To perform similarity searches on these embeddings, we used FAISS (Facebook AI Similarity Search) (Douze et al., 2025), a library designed for fast indexing and retrieval of high-dimensional vectors. FAISS's efficient similarity search capabilities meant we could keep response times low even as we expanded our metadata documentation. To further improve retrieval precision, we added a reranking step using Cohere's "rerank-english-v3.0" model (Jacob et al., 2024). This reranker applies a secondary, more detailed assessment to the top results from the initial semantic search. Specifically, it takes the top 10 retrieved chunks and reorders them according to their contextual relevance to the query, ensuring the most useful information is passed to the LLM. We also tested a sparse keyword search method, BM25, however it showed no noticeable gain when combined with the reranker. As a result, we stuck with a purely semantic search followed by reranking. Finally, the organized prompt (Eq. (10)) is passed to OpenAI's GPT-4o model, which composes the final output of the RAG component based on the retrieved metadata. This response is then passed to the downstream SQL generation step, which uses the same contextual information to generate the final SQL command.

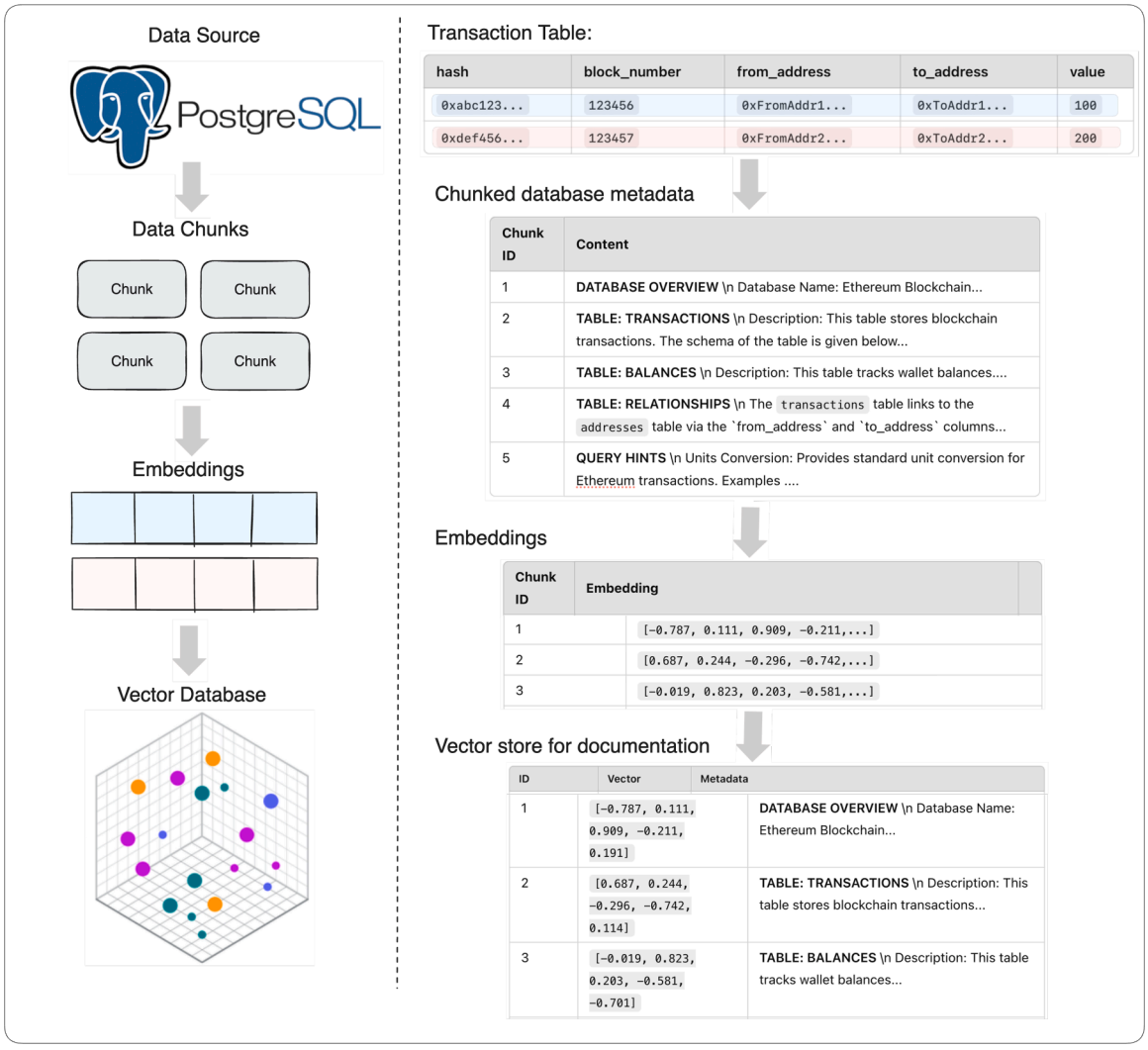


Fig. 3. RAG workflow for indexed blockchain data processing.

5.5. Query processor implementation

The Query Processor is implemented using an LLM prompted to classify incoming queries through in-context learning, as detailed in Algorithm 3. We provide the LLM with few-shot examples of various query types and their appropriate routing decisions. These examples include scenarios like “show me the latest transactions” (real-time), “analyze trading volume from last month” (historical), and “compare current gas prices with last week’s average” (hybrid), helping the LLM learn the patterns for routing decisions. Through function calling the LLM has access to current blockchain time, which it uses alongside these learned patterns to determine the appropriate processing mode.

Based on the classification result, the Query Processor routes queries to the appropriate component. Queries involving very recent data (i.e., within the hourly indexing window) are directed to the real-time component that interrogates the blockchain node directly, while older queries are served from the indexed PostgreSQL store. Hybrid requests (e.g., “show me this address’s transfers from now back to last week”) trigger parallel execution on both paths, and the merged results are synchronized before presentation.

Generative UI pipeline. In addition, the query processor formats the result for presentation to the user, including relevant blockchain data, transaction details, and explanatory text tailored to the specific query type. After merging, the processor generates a compact JSON payload containing both the extracted data and a suggested *analytics view spec*. This view spec references one or more pre-defined React components, e.g., `<BalanceCard>`, `<TxList>`, or `<TxSummary>` (Fig. 4(a)–(d)), with the appropriate props dynamically filled. Next.js renders these components server-side and streams the HTML to the browser, ensuring low-latency delivery. The components are strongly typed and pre-tested, allowing the LLM to assemble them safely and flexibly without generating raw HTML or JavaScript. By ‘generative UI,’ we refer to this dynamic composition of existing components rather than literal UI code generation. To support new query types or visualizations, developers need only register an additional React component; no changes are required in the back-end pipeline.

5.6. Evaluation dataset

To evaluate the SQL agent’s text-to-SQL capabilities, we prepared an evaluation set consisting of natural language queries paired with their corresponding correct SQL queries. Table 2 shows some samples from this dataset. The dataset contains queries of different complexity levels, with simple queries focusing on basic operations like balance lookups and single-table filtering, intermediate queries involving multi-table joins and dummyTXdummy– basic aggregations, advanced queries requiring complex analytics with multiple subqueries,

Algorithm 3: Adaptive query processor for dual-mode query routing.

```
Input: User query Q
Output: Response R
begin
    Query Classification using LLM:
        (query_type, transformed_query) ← ClassifyQueryWithLLM (Q);
        if query_type = real_time then
            mode ← Online;
    else
            mode ← Offline;
    end
    Mode Handling
    switch mode do
        case Online do
            R ← Blockchain_Agent (transforment_query, LLM, Tools);
        end
        case Offline do
            R ← SQL_Agent (transforment_query, LLM, Tools);
        end
        case Hybrid do
            R_online ← Blockchain_Agent (transforment_query, LLM, Tools);
            R_offline ← SQL_Agent (transforment_query, LLM, Tools);
            R ← ProcessWithLLM (R_online, R_offline);
        end
    end
    Optimization
        if response can be cached then
            Cache(Q, R);
    end
        Log(Q, R) for future optimization;
    Return R;
end
```

expert-level queries demanding sophisticated analysis using window functions and complex Common Table Expressions (CTEs).

The dataset is structured to evaluate different aspects of blockchain data analysis, including high-value transaction identification, mining pool pattern detection, flash loan analysis, and statistical transaction analysis. Each query in the dataset is labeled with expected or ground truth results. The dataset, for instance, includes queries to identify whale addresses with substantial ETH balances, detect contract-like behavior through transaction patterns, and analyze mining pool distribution patterns.

5.7. Evaluation method

For evaluating the SQL agent's performance in translating natural language queries into accurate SQL queries, we use Execution Accuracy (EX) (Zhong et al., 2017) and False-Less Execution (FLEX) (Kim et al., 2024). EX measures the percentage of generated SQL queries that execute correctly and return the expected results when run against the blockchain database. The FLEX metric, on the other hand, assesses whether generated SQL queries align with human expert interpretations of analytical intent regardless of syntactic variations. The authors demonstrated that FLEX scores show strong agreement with human expert judgments, significantly outperforming EX in terms of alignment with human assessments. By emphasizing semantic correctness over exact syntactic matching, FLEX reduces the likelihood of false positives or negatives arising from minor formatting issues. This metric is particularly relevant for our application, as SQL outputs undergo additional LLM processing before being visualized in the UI. Thus, semantic accuracy holds greater practical importance than syntactic precision in our context.

The evaluation process involves running each query through different configurations of the SQL Agent, including different backbone LLM models with and without RAG support, and comparing the results against the verified expected outcomes. These expected outcomes incorporate both exact matches for precise metrics and acceptable ranges for values that may fluctuate due to blockchain dynamics. The methodology accounts for various edge cases and potential failure modes, such as failed transactions, and zero-value transfers.

5.8. Performance analysis

The results in Table 3 show comparative performance of different LLMs for natural language to SQL translation in the blockchain context. Among the models tested, Claude-3.5-Sonnet performed the best, reaching a 58.62 % FLEX score when using RAG, compared to its baseline score of 51.72 %. This means it improved by 6.9 %, showing better alignment with the expected result, while there may still be variations in the output structure. GPT-4o and Gemini-2.0-Flash showed improvements of 10.34 % and 24.13 % respectively in the FLEX score, but from lower starting points. Gemini-2.0-Flash baseline had the lowest starting score with the main reason being execution failure in the generated queries. A consistent pattern we see across all models is that they perform better when RAG is integrated. The RAG system provides context-aware schema information and relevant query patterns, which leads to improved accuracy of the generated query. This is especially noticeable with Gemini-2.0-Flash, which jumped to a 51.72 % FLEX score. We also tested CHESS (Talaei et al., 2024), which is an LLM-based multi-agent text-to-sql framework. It is one of the top 20 solutions on the BIRD benchmark that supports proprietry LLMs and has the code made publicly available. We run the provided code with default prompts, GPT-4o-mini as the backend, and restricting candidate generation to a single query. It achieved a FLEX score of 44.83 %. Overall, these results indicate that even a straightforward integration of the RAG system, granting the LLM access to database metadata and domain-specific documentation, can significantly improve query generation accuracy. Fig. 5 shows an example of a user question, reformulated metadata retrieval questions, retrieved metadata, and generated corresponding SQL query. This example is representative of how the RAG pipeline operates for all queries evaluated in our experiments.

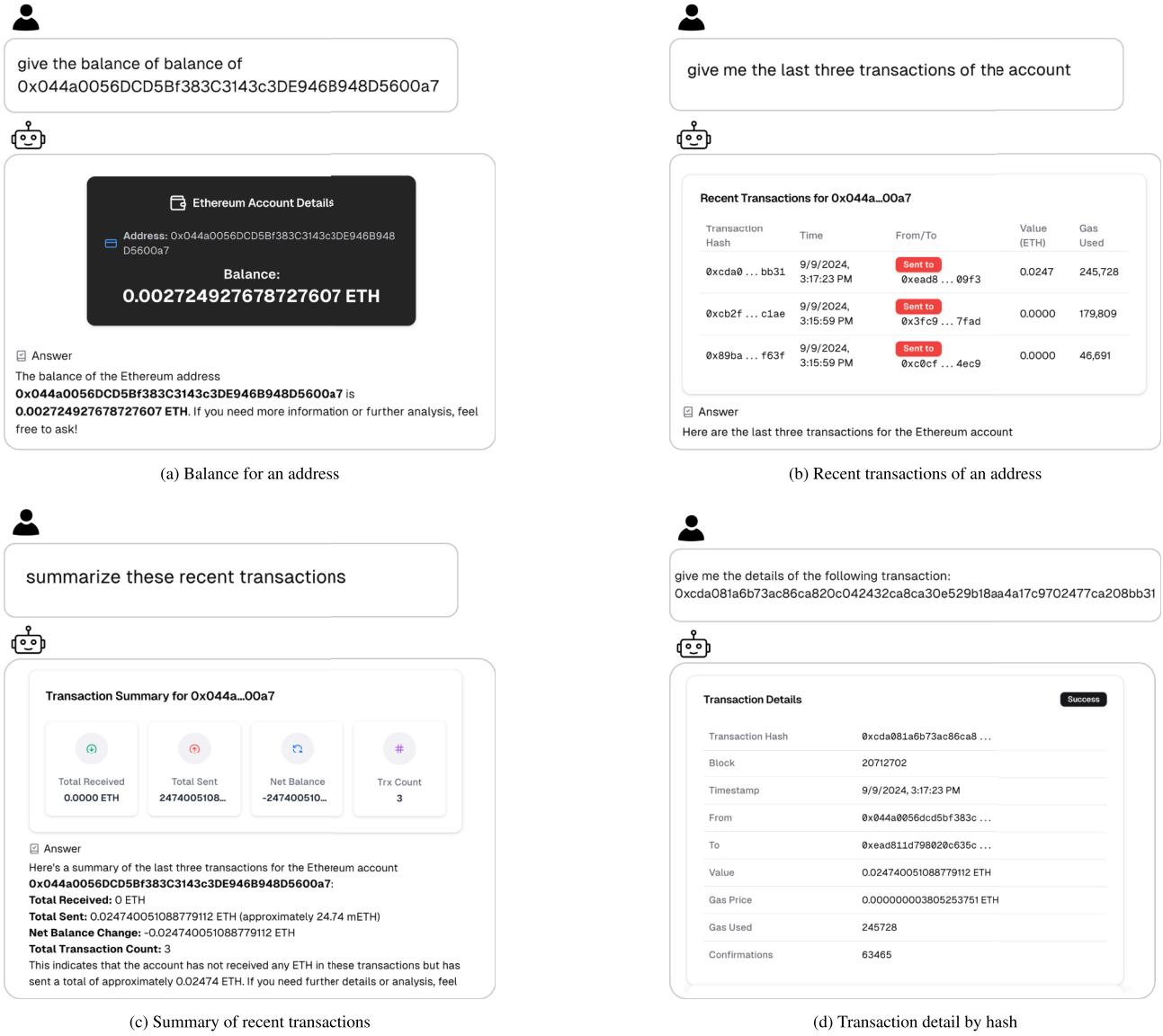
6. Results and discussion

This section presents the evaluation results of the LLM-powered blockchain explorer. It assesses the system's ability to process real-time and historical blockchain data through natural language query translation. It examines performance using a curated dataset, evaluates the impact of RAG, compares LLM models, and discusses practical implications, limitations, and generalization to other blockchain networks and structured data environments.

Detailed analysis of FLEX failures reveals two main types of errors. These were syntax errors that made the SQL queries fail to run, and also complex analytical queries that needed advanced SQL features, like window functions or multi-step aggregations. When dealing with complicated blockchain queries, such as tracking transactions over time or analyzing smart contracts, accuracy is crucial. Claude Sonnet's performance varied by query difficulty, with 57.1 % accuracy on advanced queries, 66.7 % on expert-level queries, and 100 % accuracy on intermediate and simple queries. The RAG system proves valuable in addressing both error types: it reduces syntax errors through improved database

Table 2
Sample evaluation dataset for SQL Agent, showing query complexity levels from simple lookups to expert-level blockchain analytics.

Question	SQL Query	Category	Difficulty
What is the current balance of address 0x742d35Cc6634C0532925a3b844Bc454e4438f44e?	`SELECT address, balance / 1e18 as eth_balance FROM balances WHERE address = '0x742d35Cc6634C0532925a3b844Bc454e4438f44e';`	balance_lookup	Simple
Show me all transactions where more than 100 ETH was transferred in block 15000000.	`SELECT hash, from_address, to_address, value / 1e18 as eth_value FROM transactions WHERE block_number = 15000000 AND value > 100 * 1e18;;`	transaction_filter	Simple
How many transactions occurred between blocks 15000000 and 15001000?	`SELECT COUNT(*) FROM transactions WHERE block_number BETWEEN 15000000 AND 15001000;`	transaction_count	Intermediate
Show the distribution of ETH balances in ranges (0-1, 1-10, 10-100, 100-1000, 1000+)?	`WITH balance_ranges AS (SELECT CASE WHEN balance < 1 * 1e18 THEN '0-1 ETH' WHEN balance < 10 * 1e18 THEN '1-10 ETH' WHEN balance < 100 * 1e18 THEN '10-100 ETH' WHEN balance < 1000 * 1e18 THEN '100-1000 ETH' ELSE '1000+ ETH' END as balance_range FROM balances WHERE balance > 0) SELECT balance_range, COUNT(*) as address_count FROM balance_ranges GROUP BY balance_range ORDER BY CASE WHEN balance_range = '0-1 ETH' THEN 1 WHEN balance_range = '1-10 ETH' THEN 2 WHEN balance_range = '10-100 ETH' THEN 3 WHEN balance_range = '100-1000 ETH' THEN 4 ELSE 5 END;`	balance_distribution	Intermediate
Show addresses that have exactly doubled their balance through a single transaction in the last 1000 blocks?	`WITH balance_changes AS (SELECT t.to_address, t.value, b.balance, b.balance - t.value as previous_balance FROM transactions t JOIN balances b ON t.to_address = b.address WHERE t.block_number BETWEEN 15999000 AND 16000000) SELECT to_address, value / 1e18 as received_eth, balance / 1e18 as current_balance, previous_balance / 1e18 as previous_balance FROM balance_changes WHERE previous_balance > 0 AND ABS((balance::numeric / previous_balance) - 2) < 0.001;`	balance_analysis	Advanced
What addresses have maintained a consistent balance above 1000 ETH across all transactions in blocks 15000000 to 15001000?	`WITH balance_changes AS (SELECT block_number, from_address as address, -value as change FROM transactions WHERE block_number BETWEEN 15000000 AND 15001000 UNION ALL SELECT block_number, to_address as address, value as change FROM transactions WHERE block_number BETWEEN 15000000 AND 15001000), running_balances AS (SELECT address, SUM(change) as total_change FROM balance_changes GROUP BY address) SELECT b.address, b.balance / 1e18 as current_balance, (b.balance - COALESCE(rb.total_change, 0)) / 1e18 as initial_balance FROM balances b LEFT JOIN running_balances rb ON b.address = rb.address WHERE b.balance > 1000 * 1e18 AND (b.balance - COALESCE(rb.total_change, 0)) > 1000 * 1e18;`	balance_tracking	Expert
What addresses have maintained a consistent balance above 1000 ETH across all transactions in blocks 15000000 to 15001000?	`WITH balance_changes AS (SELECT block_number, from_address as address, -value as change FROM transactions WHERE block_number BETWEEN 15000000 AND 15001000 UNION ALL SELECT block_number, to_address as address, value as change FROM transactions WHERE block_number BETWEEN 15000000 AND 15001000), running_balances AS (SELECT address, SUM(change) as total_change FROM balance_changes GROUP BY address) SELECT b.address, b.balance / 1e18 as current_balance, (b.balance - COALESCE(rb.total_change, 0)) / 1e18 as initial_balance FROM balances b LEFT JOIN running_balances rb ON b.address = rb.address WHERE b.balance > 1000 * 1e18 AND (b.balance - COALESCE(rb.total_change, 0)) > 1000 * 1e18;`	balance_tracking	Expert



(a) Balance for an address

(b) Recent transactions of an address

(c) Summary of recent transactions

(d) Transaction detail by hash

Fig. 4. Generative user interface components demonstrating: (a) Account balance display, (b) Address transaction history, (c) Transaction summary analytics, and (d) Detailed transaction information view.

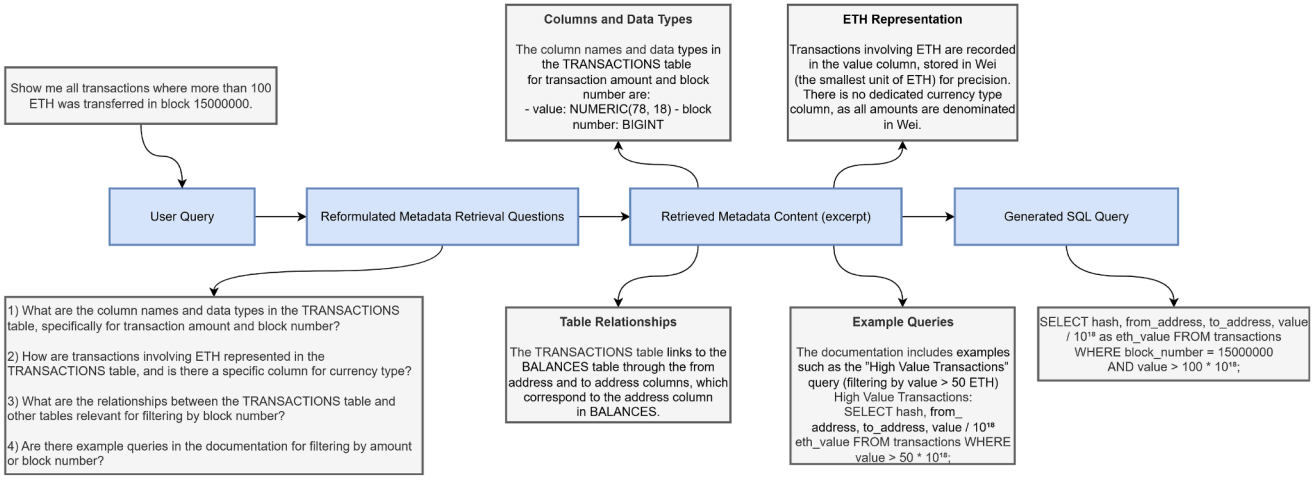


Fig. 5. Example showing user query transformation, database metadata retrieval, and generated SQL query.

Table 3
Performance comparison of different models on SQL generation with and without RAG in the offline mode.

Model	Condition	Metrics		
		FLEX	EX	EM
GPT-4o	Baseline	44.83	17.24	10.34
	+ RAG	55.17	20.69	13.79
Gemini-2.0-Flash	Baseline	27.59	13.79	10.34
	+ RAG	51.72	27.58	10.34
Claude-3.5-Sonnet	Baseline	51.72	24.14	13.79
	+ RAG	**58.62**	**31.03**	**17.24**
$CHESS_{(IR,SS,CG)}$	Default	44.83	24.13	13.79

schema alignment and assists with complex queries by providing relevant examples and structural hints.

In terms of the remaining metrics, we find that the EM scores are the lowest for all models, with even the best-performing Claude-3.5-Sonnet achieving only 17.24 % with RAG. This is expected as EM requires an exact word-for-word match with reference queries, while multiple valid query structures can express the same intent. The EX scores are higher than EM scores but still remain lower than FLEX scores across all models. This is because EX evaluation, which focuses on getting identical results, may fail to recognize semantically correct queries that produce slightly different result formats or orderings, particularly in blockchain queries involving temporal data or address aggregations. The FLEX metric is especially important in our system due to the use of the LLM-based query processor and the generative UI component. Unlike traditional blockchain explorers that directly show SQL query results on the UI, our system uses an LLM to synthesize and contextualize the results before presentation. This architectural choice means that minor variations in query structure or execution results can be interpreted and reconciled by our query processor. This makes semantic correctness, as measured by FLEX, more crucial than exact execution matches (EX) or syntactic equivalence (EM). The high FLEX scores of Claude-3.5-Sonnet suggest that it has a strong understanding of both query intent and blockchain concepts, suggesting better semantic alignment between natural language questions and generated SQL queries.

In addition to accuracy, response time is a key consideration for any user-facing system. As shown in Table 4, in tasks that involve historical data analysis, the average latency per query can be broken down into three stages: retrieving context using the RAG pipeline, generating SQL queries with the LLM, and executing those queries on the database. The results reveal a clear trade-off between speed and accuracy. Gemini-2.0-Flash is the fastest, with an average total response time of 7.93 s, followed closely by GPT-4o at 9.09 s. In contrast, Claude-3.5-Sonnet, the most accurate model, has the highest latency at 15.92 s. The primary performance bottleneck across all models is context retrieval, which accounts for 73 % to 85 % of total response time. The main reason is that each user question is expanded into 3–5 sub-queries by the model, which are then passed to the RAG system to retrieve context specifically tailored to the original query. Although the RAG system processes queries concurrently, each user question is expanded into multiple sub-queries, which introduces a slight increase in overall latency. Claude-3.5-Sonnet's higher latency, compared to the other models, is primarily due to its query generation stage, which takes 4.11 s, more than three times longer than the others. Meanwhile, database execution time is minimal and does not significantly impact the overall response.

In online mode, it typically involves the live blockchain agent making a single call to the LLM to decide which tool to use. The response time of the LLM here is the same as the "DB Query Generation" values reported in Table 4, depending on which model is used. In addition to that, there is a response latency from the JSON-RPC API call to the Ethereum gateway. In our tests, when performing *get_block* calls, the average round-trip to Infura's and PublicNode's RPC endpoints were 0.933 and 0.960 s, respectively. Other lighter calls such as balance lookup, filtering transactions through *eth_getLogs* all had sub 0.5 s latency. These latency figures, including the inference time for the different LLMs, is only indicative rather than absolute. Because both LLM inference and RPC latency can fluctuate with provider load, geographic routing, or rate-limit tiers. When we consider the hybrid mode, overall latency matches the offline figures because the SQL agent in the offline path and the live blockchain agent are executed in parallel. The latency is, therefore, dictated by the slower branch. In practice, the live agent usually returns first, the combination of one LLM call (approx. 1–4 s depending on model) plus a sub-second JSON-RPC call. So the end-to-end time is effectively bounded by the offline path's latency values reported in Table 4.

6.1. Comparison with existing solutions

Table 5 highlights the recent studies that have combined the use of LLMs and blockchain. They can generally be categorized into two camps: trust-layer proposals such as Bouchiha et al. (2024) and Luo et al. (2023), which log or audit LLM outputs on-chain but offer no querying functionality, and analytics-oriented systems such as Gai et al. (2023) and Xian et al. (2024), which operate on live transaction streams yet lack historical analysis, context retrieval from any knowledge source, or a conversational interface. ChainGPT (2025) is the only paid service that exposes a chat UI, but it is closed-source and supports neither schema-aware historical queries nor a dedicated RAG layer. In its current version, it seems to focus on market analysis and research only. In contrast, our proposed explorer is the only solution that simultaneously delivers real-time inspection, historical analytics on indexed blockchain data, hierarchical retrieval of database metadata, and a fully interactive chat front-end with generative analytics. When this functional coverage is paired with the quantitative results in Tables 3 and 4, the proposed system occupies a practical point on the accuracy-latency spectrum while closing feature gaps that exist in the other discussed solutions.

6.2. Cost analysis

Evaluating the operational cost of the proposed system is essential for assessing its practicality in real-world deployments, where LLM inference expenses can influence model selection and deployment strategy. Table 6 summarizes the average input-output token footprint of each LLM when the explorer is running in offline mode, which is then used to estimate the cost per 100 queries. In contrast, online mode queries (live chain look-ups handled by the tool-calling agent) are noticeably lighter because they do not require long database metadata or schema hints. Token usage in this mode is less than half that of the offline mode, which correspondingly reduces operational cost.

Baseline LLM calls (no RAG). Using the prices published by each model provider, the total input and output token usage counts in Table 6 are converted into a dollar cost per 100 queries, following:

$$\text{Cost per 100 queries} = \frac{(T_{\text{in}} \times P_{\text{in}}) + (T_{\text{out}} \times P_{\text{out}})}{1000000} \times 100 \tag{11}$$

where T_{in} and T_{out} are the average input and output tokens per query, and P_{in} and P_{out} are the provider's per-1M-token input and output prices in USD. The cost of GPT-4o and Claude-3.5-Sonnet in offline mode, per 100 queries, is calculated to be $0.74 and $1.10, respectively. Gemini-2.0-Flash is an order of magnitude cheaper at $0.04 for the same workload. When plotted against FLEX accuracy (Fig. 6), the results reveal a clear trade-off between semantic precision and cost: Claude delivers the highest precision, GPT-4o offers a balanced middle ground, and Gemini is the budget option for latency-sensitive dashboards.

Additional RAG overhead. As shown in Table 6, incorporating schema snippets, column descriptions, exemplar patterns, and user-level hints through the RAG system increases the average input length by 1,535 tokens and the average output length by 145 tokens per query across all models. This additional context results in higher operational costs,

Table 4
Average latency and component breakdown per query in the offline mode.

Model	Mean Latency (s)			
	Context Retrieval (RAG)	DB Query Generation	DB Execution	Total End-to-End
Gemini-2.0-Flash	**6.590**	**1.234**	0.107	**7.931**
GPT-4o	7.755	1.240	**0.092**	9.088
Claude-3.5-Sonnet	11.686	4.112	0.118	15.915

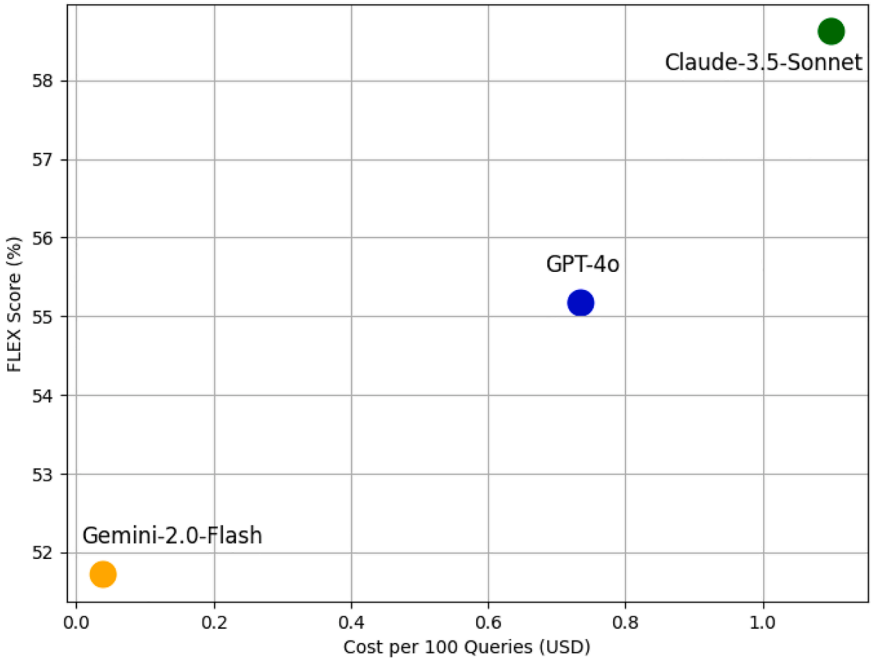


Fig. 6. Performance-cost trade-off across LLM models in offline mode.

Table 5
Comparative overview of solutions combining LLM and blockchain.

Approach	Real-time On-chain	Historical Analytics	Retrieval / RAG	Chat UI
Bouchiha et al. (2024)	×	×	×	×
Luo et al. (2023)	×	×	×	×
Gai et al. (2023)	✓	×	×	×
Xian et al. (2024)	✓	×	×	×
ChainGPT (2025)	✓	×	×	✓
This work	✓	✓	✓	✓

Table 6
Token usage and estimated cost per 100 queries for each LLM in offline mode, with and without RAG.

Model	Without RAG			With RAG		
	Input	Output	Cost/100Q (USD)	Input	Output	Cost/100Q (USD)
GPT-4o	1836	276	0.74	3371	421	1.33
Claude-3.5-Sonnet	1741	389	1.10	3276	534	1.97
Gemini-2.0-Flash	1762	317	0.04	3297	462	0.07

ranging from $0.07 per 100 queries for Gemini-2.0-Flash to $1.97 for Claude-3.5-Sonnet. Although this overhead is non-negligible, the performance gains reported in Table 3, an improvement of 10–25 percentage points in FLEX scores, suggest that the increased expenditure is often warranted in complex analytics scenarios.

6.3. Practical implications and limitations

These results have significant implications for blockchain data analysis and accessibility. First, they show that LLM-powered systems can bridge the gap between natural language and technical blockchain queries, especially when enhanced with domain-specific knowledge through RAG. Second, the performance result of Claude-3.5-Sonnet suggests that current language models are capable of handling the complexity inherent in blockchain data structures and query patterns. Lastly, the improvement across all models with RAG integration shows a clear way to enhance blockchain explorer interfaces, potentially making them more accessible to non-technical users while maintaining the capability for sophisticated analysis. Despite these positive results, there is still room for improvement. Although FLEX scores above 50 % show that LLMs can translate natural language queries effectively, execution accuracy could be better. The lower EX and EM scores indicate that in the future iteration it is important to implement agent recovery, where the SQL agent is able to regenerate queries based on the execution error message fed back in to it, or integrate schema validation layers, such as query sanitization, for better results.

A key limitation of the current implementation is its reliance on a single hosted LLM for both the live blockchain agent and the SQL generator. While this design choice simplifies deployment and ensures consistency in evaluation, it introduces a single point of failure and undermines the decentralization principle underpinning the system. To address this, a more robust architecture should incorporate a multi-provider failover mechanism, where queries are initially routed to a preferred model and automatically redirected to a secondary LLM, whether a third-party API or a self-hosted open-source alternative, if the primary service times out or degrades.

In scenarios where a self-hosted approach is preferred, performance comparable to the proprietary LLMs used in our experiments would typically require open-source models exceeding 14 billion parameters. Running such models locally demands high-VRAM GPUs (e.g., NVIDIA A100 or H100) or equivalent cloud-based GPU instances. While this enables full control over the inference process and eliminates API-related

costs, it may involve substantial upfront or operational expenses, potentially outweighing the savings for certain workloads. The choice between hosted and local deployment therefore depends on the specific cost-performance trade-offs, hardware availability, and privacy requirements of the target application domain. A lightweight, gated ensemble that selects the fastest model meeting a minimum quality threshold could further improve system resilience and responsiveness in either deployment mode.

6.4. Generalization

Our LLM-based blockchain explorer can be adapted for different blockchain networks other than Ethereum. The system's core components, particularly the schema-aware SQL agent and RAG pipeline, are blockchain-agnostic by design. This means we can replace the database structure used while indexing Ethereum transactions with those suitable for other blockchains. Similarly, for high-throughput chains like Solana or BNB Chain, the real-time monitoring component can be adjusted to handle micro-block granularity by configuring the system's time-window parameter for real-time transactions. Another direction for generalization is enabling multi-chain query federation. This would allow users to perform comparative analysis across different blockchain networks through federated queries. This capability would probably require unified data model that connects different blockchain platforms, making it easier to analyze multiple networks at once.

Beyond blockchain applications, our approach of translating natural language to schema-grounded queries has broad applicability to other structured data domains. The framework could serve as a universal SQL interface for traditional enterprise databases, such as medical records or supply chain logs, provided appropriate schema documentation is available for RAG retrieval. The framework could work as an AI-powered SQL interface for traditional databases, such as medical records or supply chain logs, provided appropriate schema documentation is available for RAG retrieval. Furthermore, integration with Web3 technologies like IPFS for decentralized file storage or blockchain oracles for off-chain data feeds would enable even more complex compound queries that take in off-chain data into consideration.

7. Conclusion

In this paper, we presented an innovative approach to blockchain data exploration through a natural language interface powered by LLMs. Our system addressed the fundamental challenges of blockchain data accessibility by combining real-time blockchain interaction capabilities with advanced historical data analysis. We designed a hybrid architecture that integrates an LLM-powered blockchain agent for real-time queries with a schema-aware SQL agent for historical analysis. By incorporating a RAG system, we enhanced the SQL agent's understanding of database schema leading to more accurate database query generation. In our evaluation, we tested a diverse set of blockchain queries and demonstrated several key strengths of the proposed system. First, the schema-aware SQL agent showed remarkable effectiveness in translating natural language queries into precise SQL commands, which enabled accurate retrieval of historical blockchain data. Second, the LLM-powered blockchain agent showed the ability to handle real-time blockchain interactions providing access to current blockchain states. The query processor successfully routed queries between these processing paths for different types of blockchain data requests. The performance in handling both simple lookups and complex analytical queries suggested that our approach successfully bridges the gap between technical blockchain data structures and user-friendly exploration interfaces. In future work, we plan to add autonomous action agents that take analyzed data and execute optimized transactions and interact with smart contracts based on natural language instructions.

CRediT authorship contribution statement

S. Gebreab: Formal analysis, Methodology, Investigation, Software, Writing – original draft; **A. Musamih:** Formal analysis, Methodology, Validation, Writing – review & editing; **K. Salah:** Conceptualization, Methodology, Funding acquisition, Project administration, Supervision, Validation, Writing – review & editing; **R. Jayaraman:** Validation, Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

The authors sincerely thank Prof. K. Selçuk Candan, Director of CASCADE, School of Computing and Augmented Intelligence, Arizona State University, for his valuable feedback and suggestions that helped improve the technical rigor of this work. This publication is based upon work supported by the Khalifa University of Science and Technology under Award No. RIG-2023-049. We acknowledge that Claude, AI assistant, was used to improve the English language of the paper.

References

Bharathi Mohan, G., Prasanna Kumar, R., & Vishal Krishh, P., Keerthinathan, A., Lavanya, G., Meghana, Meka Kavya Uma, Sulthana, Sheba, & Doss, Srinath (2024). An analysis of large language models: Their impact and potential applications. *Knowledge and Information Systems*, *66* (9), 5047–5070. https://doi.org/10.1007/s10115-024-02120-8

Bouchiha, M. A., Telnoff, Q., Bakkali, S., Champagnat, R., Rabah, M., Coustaty, M., & Ghamri-Doudane, Y. (2024). LLMChain: Blockchain-based reputation system for sharing and evaluating large language models. https://arxiv.org/abs/2404.13236.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., & Amodei, D. (2020). Language models are few-shot learners. https://arxiv.org/abs/2005.14165.

Buterin, V. (2014). A next-generation smart contract and decentralized application platform. White paper. https://ethereum.org/en/whitepaper/.

ChainGPT (2025). AIVM (artificial intelligence virtual machine) whitepaper: The layer-1 infrastructure for a transparent, scalable ai economy. Accessed: 2025-06-23 https://tinyurl.com/2pn4t6a7.

Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., Schuh, P., Shi, K., Tsvyashchenko, S., Maynez, J., Rao, A., Barnes, P., Tay, Y., Shazeer, N., Prabhakaran, V., Reif, E., Du, N., Hutchinson, B., Pope, R., Bradbury, J., Austin, J., Isard, M., Gur-Ari, G., Yin, P., Duke, T., Levskaya, A., Ghemawat, S., Dev, S., Michalewski, H., Garcia, X., Misra, V., Robinson, K., Fedus, L., Zhou, D., Ippolito, D., Luan, D., Lim, H., Zoph, B., Spiridonov, A., Sepassi, R., Dohan, D., Agrawal, S., Omernick, M., Dai, A. M., Pillai, T. S., Pellat, M., Lewkowycz, A., Moreira, E., Child, R., Polozov, O., Lee, K., Zhou, Z., Wang, X., Saeta, B., Diaz, M., Firat, O., Catasta, M., Wei, J., Meier-Hellstern, K., Eck, D., Dean, J., Petrov, S., & Fiedel, N. (2023). PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research*, *24*(240), 1–113. http://jmlr.org/papers/v24/22-1144.html.

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., & Jégou, H. (2025). The Faiss library. https://arxiv.org/abs/2401.08281.

Gai, Y., Zhou, L., Qin, K., Song, D., & Gervais, A. (2023). Blockchain large language models. https://arxiv.org/abs/2304.12749.

Gallifant, J., Fiske, A., Levites Strekalova, Y. A., Osorio-Valencia, J. S., Parke, R., Mwavu, R., Martinez, N., Gichoya, J. W., Ghassemi, M., Demner-Fushman, D., McCoy, L. G., Celi, L. A., & Pierce, R. (2024). Peer review of GPT-4 technical report and systems card. *PLOS Digital Health*, *3* (1), 1–15. https://doi.org/10.1371/journal.pdig.0000417. https://doi.org/10.1371/journal.pdig.0000417

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-augmented generation for large language models: A survey. https://arxiv.org/abs/2312.10997.

Gebreab, S. A., Salah, K., Jayaraman, R., Rehman, M. H. U., & Ellaham, S. (2024). LLM-based framework for administrative task automation in healthcare. In *2024 12th International symposium on digital forensics and security (ISDFS)* (pp. 1–7). https://doi.org/10.1109/ISDFS60797.2024.10527275

Gemini Team, Google (2024). Gemini: A family of highly capable multimodal models. https://arxiv.org/abs/2312.11805.

Geren, C., Board, A., Dagher, G. G., Andersen, T., & Zhuang, J. (2024). Blockchain for large language model security and safety: A holistic survey. https://arxiv.org/abs/2407.20181.

Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). REALM: Retrieval-augmented language model pre-training. https://arxiv.org/abs/2002.08909.

He, Z., Li, Z., Yang, S., Ye, H., Qiao, A., Zhang, X., Luo, X., & Chen, T. (2025). Large language models for blockchain security: A systematic literature review. https://arxiv.org/abs/2403.14280.

Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. https://arxiv.org/abs/2007.01282.

Jacob, M., Lindgren, E., Zaharia, M., Carbin, M., Khattab, O., & Drozdov, A. (2024). Drowning in documents: Consequences of scaling reranker inference. https://arxiv.org/abs/2411.11767.

Juba, S., Vannahme, A., & Volkov, A. (2015). Learning postgreSQL. Community experience distilled. Packt Publishing. https://books.google.ae/books?id=aE54jwEACAAJ.

Kamalloo, E., Zhang, X., Ogundepo, O., Thakur, N., Alfonso-Hermelo, D., Rezagholizadeh, M., & Lin, J.(2023). Evaluating embedding APIs for information retrieval. https://arxiv.org/abs/2305.06300.

Kim, H., Jeon, T., Choi, S., Choi, S., & Cho, H. (2024). FLEX: Expert-level false-less execution metric for reliable text-to-SQL benchmark. https://arxiv.org/abs/2409.19014.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive NLP tasks. https://arxiv.org/abs/2005.11401.

Luo, H., Luo, J., & Vasilakos, A. V. (2023). BC4LLM: Trusted artificial intelligence when blockchain meets large language models. https://arxiv.org/abs/2310.06278.

Meyer, E. A., Welpe, I. M., & Sandner, P. (2022). Decentralized finance – A systematic literature review and research directions. In *ECIS 2022 research papers* 25. https://aisel.aisnet.org/ecis2022_rp/25. https://doi.org/10.2139/ssrn.4016497

Monrat, A. A., Schelén, O., & Andersson, K. (2019). A survey of blockchain from the perspectives of applications, challenges, and opportunities. *IEEE Access*, *7*, 117134–117151. https://doi.org/10.1109/ACCESS.2019.2936094

Nakamoto, S. (2008). Bitcoin: A peer-to-peer electronic cash system. White paper. https://bitcoin.org/bitcoin.pdf.

Pourreza, M., Li, H., Sun, R., Chung, Y., Talaei, S., Kakkar, G. T., Gan, Y., Saberi, A., Ozcan, F., & Arik, S. O. (2024). CHASE-SQL: Multi-path reasoning and preference optimized candidate selection in Text-to-SQL. https://arxiv.org/abs/2410.01943.

Shi, L., Tang, Z., Zhang, N., Zhang, X., & Yang, Z. (2024). A survey on employing large language models for text-to-SQL tasks. https://arxiv.org/abs/2407.15186.

Talaei, S., Pourreza, M., Chang, Y.-C., Mirhoseini, A., & Saberi, A. (2024). CHESS: Contextual harnessing for efficient SQL synthesis. https://arxiv.org/abs/2405.16755.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., & Lample, G. (2023). Llama: Open and efficient foundation language models. https://arxiv.org/abs/2302.13971.

Tovanich, N., Heulot, N., Fekete, J.-D., & Isenberg, P. (2021). Visualization of blockchain data: A systematic review. *IEEE Transactions on Visualization and Computer Graphics*, *27* (7), 3135–3152. https://doi.org/10.1109/TVCG.2019.2963018

Toyoda, K., Wang, X., Li, M., Gao, B., Wang, Y., & Wei, Q. (2024). Blockchain data analysis in the era of large-language models. https://arxiv.org/abs/2412.09640.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, *18* (6), 186345. https://doi.org/10.1007/s11704-024-40231-1

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E. H., Le, Q. V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NeurIPS)* (pp. 1800–1813).

Werner, S., Perez, D., Gudgeon, L., Klages-Mundt, A., Harz, D., & Knottenbelt, W. (2023). SoK: Decentralized finance (DeFi). AFT '22 (p. 30–46). New York, NY, USA: Association for Computing Machinery. https://doi.org/10.1145/3558535.3559780

Wood, G. (2014). Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper*, . https://ethereum.github.io/yellowpaper/paper.pdf.

Xian, Y., Zeng, X., Xuan, D., Yang, D., Li, C., Fan, P., & Liu, P. (2024). Connecting large language models with blockchain: Advancing the evolution of smart contracts from automation to intelligence. https://arxiv.org/abs/2412.02263.

Xu, M., Chen, X., & Kou, G. (2019). A systematic review of blockchain. *Financial Innovation*, *5* (1), 27. https://doi.org/10.1186/s40854-019-0147-z

Zhao, W. X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., Du, Y., Yang, C., Chen, Y., Chen, Z., Jiang, J., Ren, R., Li, Y., Tang, X., Liu, Z., Liu, P., Nie, J. Y., & Wen, J. R. (2025). A survey of large language models. https://arxiv.org/abs/2303.18223.

Zhong, V., Xiong, C., & Socher, R. (2017). Seq2SQL: GGenerating structured queries from natural language using reinforcement learning. https://arxiv.org/abs/1709.00103.

Zhu, Y., Li, J., Li, G., Zhao, Y., Li, J., Jin, Z., & Mei, H. (2023). Hot or cold? Adaptive temperature sampling for code generation with large language models. https://arxiv.org/abs/2309.02772.